



Department of Computer Science & Engineering Assignment

Course Name : Structured Programming

Course Code : CSE 1383

Title : Medicine stock and expiry management for a community health center

Summer 2026

Submitted to

Abdullah Mohammad Sakib

Lecturer

Submitted by

Mohammed Khaled Bin Ibrahim

ID: 26103014

Department of Computer Science & Section: E Engineering

Department of Computer Science &
Engineering

Date of Submission: 06/09/2026

[Type here]

1. Introduction

A community health center needs a continuous and reliable supply of essential medicines to treat the patients who visit it. The amount of each medicine in the store does not stay the same. It changes almost every day because medicines are given to patients, sometimes issued in large quantities, and restocked when new supplies arrive. Since the health center usually has limited storage space, a limited budget, and a certain amount of time before new supplies arrive, it is important to regularly monitor the stock of each medicine. Good stock management can directly affect the quality of care that patients receive.

Poor stock management can lead to two main problems. First, keeping very large amounts of every medicine can reduce the chance of running out of stock. However, some of the medicines may not be used before they expire. This can lead to wasted medicines, wasted money, and wasted storage space. On the other hand, keeping only small amounts of medicine can reduce the chance of medicines expiring, but it also increases the risk of running out. A sudden increase in the number of patients or a delay in receiving the next supply could leave the health center without an important medicine when a patient needs it.

Another important issue is that not all medicines have the same level of importance. Some medicines are essential for patient care and may not have an easy substitute. These medicines should not be allowed to run out, even for a short time. A short shortage of a less important medicine may not have the same serious effect. Also, a medicine may appear to have enough stock, but it could still need attention if a large part of it is close to its expiry date and is unlikely to be used before then.

1.2 Objective of the Developed Program

The main objective of this assignment is to design and implement a structured program in the C programming language for managing medicine stock in a community health center. The program will check the current stock of medicines, identify medicines that may have a shortage risk, an expiry risk, or both, and decide which medicines should receive attention first. It will also create an organized report and save it in a file so that the health center management can use the information to make decisions.

The program should not only work with the six medicines given in this assignment. It should also be able to work with other similar sets of medicine records. The values used in the example are only for demonstration and should not be treated as fixed values.

2. Problem Analysis

The dataset supplied for the purpose of this assignment, and used throughout this report for illustration, is reproduced below.

Medicine Code	Medicine Name	Available Stock	Avg. Daily Requirement	Minimum Stock Level	Days Remaining to Expiry	Essentiality Level
---------------	---------------	-----------------	------------------------	---------------------	--------------------------	--------------------

MED01	Oral Saline	120	35	80	45	3
MED02	Paracetamol	300	40	100	120	2
MED03	Insulin	60	12	50	30	3
MED04	Amoxicillin	200	25	80	20	3
MED05	Antacid	250	18	60	15	1
MED06	Cetirizine	180	15	50	90	1

2.1 Major Requirements

- Represent each medicine using its code, name, available stock, average daily requirement, minimum stock level, days remaining before expiry, and essentiality level. These seven details should always stay correctly connected to the right medicine during all operations.
- Calculate additional information from the stored data when needed. One important value is the estimated number of days the current stock will last based on the average daily requirement.
- Classify each medicine according to its current stock and expiry situation. The program should use different categories instead of giving every medicine the same general status.
- Rank the medicines based on how urgently they need attention. If two medicines have the same urgency level, the program should use a reasonable method to decide their order.
- Search for a particular medicine using an identifying value, such as its code, and display its complete current status.
- Update the stock when new medicine is received or when medicine is given to a patient. The program must not allow the stock amount to become negative, and the medicine's status should be updated immediately after the stock changes.
- Generate a final organized report showing the medicines and their current conditions. The report should also include a summary of how many medicines are in each category and should be saved in a file for future reference.

2.2 Stock-Related Constraints

The available stock of each medicine should be checked using two important values: the minimum stock level and the average daily requirement. The minimum stock level shows the point below which the medicine is considered to be under-stocked. The average daily requirement shows how quickly the medicine is normally used.

A medicine may be below its minimum stock level but still have enough stock to last for several days if it is used slowly. On the other hand, a medicine may be above its minimum stock level but still be at risk of running out soon if it is used quickly. Therefore, both the minimum stock level and the average daily requirement need to be considered together. The program should not make decisions based only on the current stock amount.

2.3 Expiry-Related Constraints

The number of days remaining before expiry shows how much time is left before the current batch of medicine can no longer be safely used. Having a large amount of medicine in stock does not always mean that the stock level is good. If the medicine is close to its expiry date and is not being used quickly enough, some of it may expire before it can be used.

Therefore, the program should estimate whether the current stock of each medicine can be used before its expiry date based on its average daily requirement. If some of the stock is not expected to be used in time, that amount should be considered at risk of wastage. This expiry risk should be checked separately from the minimum stock level, because a medicine can have both an expiry risk and a shortage risk at the same time.

2.4 Conflicting Requirements

Some of the requirements of the program can conflict with each other. This means that the solution cannot focus on only one goal. It needs to find a reasonable balance between the different requirements. The main conflicts are shown below.

Requirement	Conflicting Requirement
Maintain sufficient medicine stock	Avoid excessive stock and expiry-related wastage
Replenish low-stock medicine	Avoid unnecessary replenishment when existing stock may expire
Give priority to essential medicine	Attend to other medicines having immediate expiry desk
Maintain a simple decision method	Consider sufficient information for a reliable decision
Process stock information accurately	Avoid unnecessary repeated calculations and processing

The proposed solution will try to balance each of these requirements instead of focusing on only one of them. This should help the program make practical and reliable decisions about the medicine stock.

3. Proposed Solution

3.1 Method Used to Analyze Stock

For each medicine, the program calculates an estimated stock-coverage period in days. This is found by dividing the available stock by the average daily requirement. This value is important because it shows how many days the current stock is expected to last. It can then be compared with the minimum stock level and the number of days left before the medicine expires.

The stock-coverage period is calculated once using the current medicine information. The program then uses this value whenever it is needed instead of calculating it again and again. This makes the program more efficient and follows the optimization requirement of the assignment.

The program also calculates how much of the medicine is expected to be used before it expires. This is done by multiplying the average daily requirement by the number of days remaining to expiry. The expected consumption is then compared with the available stock. If the available stock is greater than the expected consumption, some of the medicine may remain unused and expire. This helps the program identify medicines that are at risk of wastage due to expiry.

3.2 Method Used to Identify Stock Condition

Rather than relying on the available stock figure in isolation, the proposed classification considers, jointly, the relationship of the available stock to the minimum stock level, the estimated stock-coverage period relative to the essentiality level of the medicine, and the estimated expiry exposure derived above. On this combined basis, each medicine is assigned to one of the following five conditions.

Stock Condition	Governing Situation
Sufficient Stock	The available stock is at or above the minimum stock level. The stock is expected to last until the next supply arrives, and there is no major expiry risk.
Reorder Required	The stock is below the minimum level, or the stock-coverage period is short. However, the medicine is not highly essential and there is no immediate risk to patients.
Urgent Reorder	The stock is below the minimum level for a highly essential medicine, or the stock is expected to run out before a new supply can arrive.
Expiry Attention	The stock is enough to meet the minimum level, but a large amount is not expected to be used before the expiry date. This creates a risk of wastage.
Critical Condition	The medicine has both a shortage risk and an expiry risk, or it is an essential medicine with a short stock-coverage period. It needs immediate attention from management.

The distinguishing feature of this classification, compared with a scheme based on the available stock alone, is that it separates a medicine that is genuinely short of stock from a medicine that merely appears well-stocked but is exposed to wastage, and it additionally recognizes that a medicine can display both concerns at once, in which case it is placed in the Critical Condition category rather than under either concern alone.

3.3 Method Used to Determine Management Priority

Because the center cannot address every outstanding stock concern simultaneously, the medicines must be placed in an order that reflects the urgency of managerial attention each one requires. The proposed method combines the factors listed below into a single, comparable priority indicator for every medicine, rather than ranking the medicines on any one factor alone.

Factor	Symbol	Interpretation in the Priority Score
Essentiality Level	E	A higher essentiality level gives a higher priority score because a shortage of an essential medicine can have a greater effect on patient care.
Factor	Symbol	Interpretation in the Priority Score
Shortage Severity	S	Shows how far the available stock is below the minimum stock level. A medicine that is already below the minimum level gets a higher score than one that is only close to the minimum.
Stock Coverage (days)	C	Shows how many days the current stock is expected to last based on average daily use. A shorter coverage period means higher urgency.
Expiry Urgency	X	Based on the number of days left before expiry. A medicine that is close to expiry gets a higher score, especially if it also has a shortage risk.
Combined Risk Indicator	R	Gives an extra increase to the priority score when a medicine has both shortage risk and expiry risk at the same time.

A medicine of high essentiality that is already below its minimum stock level, and whose estimated coverage period is short, receives the highest combined priority indicator and is therefore placed at the top of the management-attention order. A medicine that shows only a mild expiry exposure, with sufficient stock and lower essentiality, receives a comparatively low priority indicator and is placed toward the bottom of the order. Because the priority indicator for a given medicine depends only on values that have already been derived while analyzing that medicine's stock condition, it is computed directly from those stored, previously calculated results rather than by re-examining the raw data a second time, again in keeping with the optimization requirement.

3.4 Method Used to Resolve Equal-Priority Cases

Where two or more medicines produce an identical, or near-identical, combined priority indicator, the proposed solution resolves the tie through a short sequence of secondary comparisons applied in order: first, the medicine with the higher essentiality level is placed ahead; second, if the essentiality levels are also equal, the medicine with the shorter estimated stock-coverage period is placed ahead, since it faces the more immediate risk of running out; and third, if the coverage periods are also equal, the medicine with fewer days remaining to expiry is placed ahead, since its exposure window is narrower. This sequence of tie-breaking comparisons is deterministic, requires no additional data beyond what has already been derived, and reflects the same clinical reasoning applied to the primary ranking itself.

3.5 Handling Stock and Expiry Conflict

The assignment requires the program to avoid automatically recommending more stock whenever a medicine appears to be low. This is important because a medicine may have low stock while also having some existing stock that is close to expiry. To handle this properly, the program keeps the shortage risk and expiry risk as two separate conditions instead of treating them as one general problem.

A medicine is considered to have a shortage risk based on its stock coverage compared with the minimum stock level. It is considered to have an expiry risk based on how much of the current stock is expected to be used before the expiry date. A medicine is considered to have a combined risk only when both conditions are present.

Keeping these risks separate allows the final report to clearly show what problem each medicine has. This helps management avoid ordering more medicine when the main problem is that the existing stock may expire unused. It also makes sure that medicines with both low stock and expiry risk are not overlooked, as these cases require more urgent attention.

4. Program Design

4.1 Data Organization

Each medicine is represented as a single structured record containing its code, name, available stock, average daily requirement, minimum stock level, days remaining to expiry and essentiality level as supplied, together with derived fields for the estimated stock-coverage period, the estimated expiry exposure, the assigned stock condition and the computed priority indicator. The complete set of medicine records is held in a single array of such structures, sized to accommodate the supplied dataset while remaining straightforward to extend if the number of medicines changes, in line with the requirement that the program continue to operate correctly when the supplied values are changed. Holding the derived fields alongside the supplied fields, within the same record, avoids the need for separate parallel arrays and ensures that every value associated with a given medicine moves together whenever the array is rearranged, searched or updated.

4.2 Major Functions

The proposed solution is organized into a small number of clearly separated functions, each responsible for a single aspect of the overall task, so that repeated operations are implemented once and reused wherever they are required. The intended responsibility of each function is summarized below; the corresponding source code is presented separately as part of the submitted implementation.

Proposed Function (Purpose)	Responsibility
Data initialization routine	Loads the medicine records, including code, name, available stock, average daily requirement, minimum stock level, days remaining to expiry, and essentiality level, when the program starts. It should be able to handle the six medicines given in the assignment as well as datasets of different sizes.
Stock-coverage calculator	Calculates how many days the available stock is expected to last based on the average daily requirement. The result is stored so that the program does not need to calculate it repeatedly.
Expiry-exposure calculator	Calculates how much medicine is expected to remain unused when the medicine reaches its expiry date. This helps the program identify expiry risk separately from shortage risk.
Stock-condition classifier	Uses the stock coverage, expiry information, minimum stock level, and essentiality level to place each medicine into one of the defined stock-condition categories.
Proposed Function (Purpose)	Responsibility
Priority evaluator	Combines essentiality, shortage severity, stock coverage, and expiry urgency to calculate a priority score for each medicine. It also handles medicines that have the same priority score using the secondary comparison rules.
Search routine	Allows the user to search for a medicine using its code or name. If the medicine cannot be found, the program displays a suitable message.
Ordering routine	Sorts the medicine records according to the selected criterion, such as management priority. It keeps all the information belonging to each medicine correctly connected while sorting.
Stock-update routine	Updates the available stock when new medicine is received or when medicine is given to patients. It does not allow the stock to become negative and recalculates the necessary values after an update.

Report generator	Creates a clear final report showing the important details of each medicine, such as its stock level, stock coverage, expiry risk, stock condition, and priority score. It also shows the total number of medicines in each stock-condition category. The report is displayed on the screen and saved to a file so that management can review it later.
Menu / control routine	Shows the user the available program options, accepts their choice, and calls the required function. This allows the program to be used multiple times without restarting it

4.3 Algorithm / Pseudocode

The overall flow of the program can be described as follows. It explains how the program will work without going into the actual C programming syntax.

1. Load the medicine records into an array of structures when the program starts.
2. Process each medicine in one pass through the array. For every medicine, the program calculates the stock-coverage period, expiry exposure, stock condition, and priority score. These calculated values are stored in the medicine record so they do not need to be calculated again later.
3. Display the main menu to the user. The menu will provide options such as searching for a medicine, updating stock, viewing medicines by priority, and generating the final report.
4. Search for a medicine: If the user chooses the search option, they enter a medicine code or name. The program searches the array and displays all the stored and calculated information if a match is found. If there is no match, an appropriate message is displayed.
5. Update medicine stock: If the user chooses the update option, they select the medicine and specify whether new stock has been received or medicine has been issued to patients. The program updates the available stock without allowing it to become negative. It then recalculates the stock coverage, expiry exposure, stock condition, and priority score only for that medicine.
6. Order the medicines: If the user chooses the ordering option, the program sorts the medicines using the selected criterion, such as the priority score. If two medicines have the same priority, the program uses the specified tie-breaking rules. All information belonging to each medicine stays together while sorting.
7. Generate the report: If the user chooses the report option, the program goes through the medicine records and displays the code, name, available stock, stock coverage, days remaining to expiry, essentiality level, stock condition, and priority score. The same report is also saved to an output file. During this process, the program counts the medicines that require reordering, urgent attention, or have expiry concerns. It also identifies the medicine with the highest priority.
8. Return to the menu: After completing an operation, the program returns to the main menu. It continues doing this until the user chooses to exit the program.

4.4 Flowchart Description

The corresponding flow proceeds, at a high level, from program start, through the one-time loading and analysis of the dataset described in Step 2 above, into a repeating menu loop that offers the search, update, ordering and reporting operations as separate branches. Each branch performs its operation and then returns control to the menu, so that the loading and initial analysis step is executed exactly once, while the operations that depend on it may be exercised repeatedly, including by the course instructor during demonstration, without repeating the initial analysis unnecessarily. The loop terminates only when the user selects the exit option, at which point the program ends.

5. Optimization Considerations

The proposed design applies the optimization requirement of the assignment in the following specific ways.

- **Single-pass calculation:** The program calculates the stock coverage, expiry exposure, stock condition, and priority score for each medicine once after loading the data. These values are stored in the medicine record instead of being recalculated every time the information is displayed, searched, sorted, or reported.
- **Recalculation only when needed:** When the stock of one medicine is changed, the program recalculates only the derived values for that medicine. The other medicines are not recalculated because their information has not changed.
- **Reuse of search results:** Once the program finds a medicine during a search, it can use the position of that medicine in the array for the next operation. This avoids searching for the same medicine again during the same interaction.
- **Single data structure:** All the information about a medicine, including both the original and calculated values, is kept in one structure. This is better than using separate arrays because all the medicine details stay together and there is less chance of the data becoming inconsistent.
- **Single-pass report generation:** The program calculates the summary information, such as the number of medicines needing reorder, urgent attention, or having expiry concerns, while it is already going through the array to create the report. This avoids making another separate pass through the data.
- **Suitable sorting method:** Since the program is expected to handle a small number of medicine records, a simple comparison-based sorting method is enough. Using a more complicated sorting method would not be necessary for such a small dataset.

6. code

```

Start here X *MedicineX X
1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAX 50
5
6  struct Medicine
7  {
8      char code[10];
9      char name[30];
10     int stock;
11     int daily;
12     int minimum;
13     int expiry;
14     int essential;
15 };
16
17 struct Medicine med[MAX] =
18 {
19     {"MED01", "Oral Saline", 120, 35, 80, 45, 3},
20     {"MED02", "Paracetamol", 300, 40, 100, 120, 2},
21     {"MED03", "Insulin", 60, 12, 50, 30, 3},
22     {"MED04", "Amoxicillin", 200, 25, 80, 20, 3},
23     {"MED05", "Antacid", 250, 18, 60, 15, 1},
24     {"MED06", "Cetirizine", 180, 15, 50, 90, 1}
25 };
26
27 int n = 6;
28
29 /* Calculate how many days the stock will last */
30 float coverage(struct Medicine m)
31 {
32     if (m.daily == 0)
33         return 0;
34     return (float)m.stock / m.daily;
35 }
36
37 /* Check medicine condition */
38 void condition(struct Medicine m)
39 {
40     float days = coverage(m);
41     if (m.stock < m.minimum && m.expiry <= 30)
42         printf("Critical");
43     else if (m.stock < m.minimum)

```

```

49     printf("Reorder");
50
51     else if (m.expiry <= 30)
52         printf("Expiry Attention");
53
54     else
55         printf("Sufficient");
56 }
57
58 /* Find medicine by code */
59 int searchMedicine(char code[])
60 {
61     int i;
62     for (i = 0; i < n; i++)
63     {
64         if (strcmp(med[i].code, code) == 0)
65             return i;
66     }
67     return -1;
68 }
69
70 /* Calculate priority */
71 int priority(struct Medicine m)
72 {
73     int score = m.essential * 20;
74     if (m.stock < m.minimum)
75         score = score + 30;
76     if (coverage(m) <= 3)
77         score = score + 25;
78     else if (coverage(m) <= 7)
79         score = score + 15;
80     else if (coverage(m) <= 14)
81         score = score + 5;
82     if (m.expiry <= 7)
83         score = score + 25;
84     else if (m.expiry <= 14)
85         score = score + 15;
86     else if (m.expiry <= 30)
87         score = score + 5;
88     return score;
89 }

```

```

97 }
98
99
100 /* Display one medicine */
101 void showMedicine(int i)
102 {
103     printf("\nMedicine Code: %s", med[i].code);
104     printf("\nMedicine Name: %s", med[i].name);
105     printf("\nStock: %d", med[i].stock);
106     printf("\nDaily Requirement: %d", med[i].daily);
107     printf("\nMinimum Stock: %d", med[i].minimum);
108     printf("\nCoverage: %.2f days", coverage(med[i]));
109     printf("\nExpiry Days: %d", med[i].expiry);
110     printf("\nEssentiality: %d", med[i].essential);
111     printf("\nCondition: ");
112     condition(med[i]);
113     printf("\nPriority: %d\n", priority(med[i]));
114 }
115
116
117 /* Display all medicines */
118 void display()
119 {
120     int i;
121
122     printf("\n\nCODE\tNAME\t\tSTOCK\tDAILY\tMIN\tEXPIRY\tPRIORITY");
123
124     for (i = 0; i < n; i++)
125     {
126         printf("\n%s\t%-15s\t%d\t%d\t%d\t%d\t%d",
127             med[i].code,
128             med[i].name,
129             med[i].stock,
130             med[i].daily,
131             med[i].minimum,
132             med[i].expiry,
133             priority(med[i]));
134     }
135
136     printf("\n");
137 }
138
139
140 /* Search and show medicine */
141 void search()
142 {
143     char code[10];
144     int index;

```

```

145
146     printf("\nEnter medicine code: ");
147     scanf("%s", code);
148
149     index = searchMedicine(code);
150
151     if (index == -1)
152         printf("Medicine not found.\n");
153     else
154         showMedicine(index);
155 }
156
157
158 /* Add or issue stock */
159 void update()
160 {
161     char code[10];
162     int index;
163     int choice;
164     int amount;
165
166     printf("\nEnter medicine code: ");
167     scanf("%s", code);
168
169     index = searchMedicine(code);
170
171     if (index == -1)
172     {
173         printf("Medicine not found.\n");
174         return;
175     }
176
177     printf("\n1. Add Stock");
178     printf("\n2. Issue Medicine");
179     printf("\nEnter choice: ");
180     scanf("%d", &choice);
181
182     printf("Enter amount: ");
183     scanf("%d", &amount);
184
185     if (amount < 0)
186     {
187         printf("Invalid amount.\n");
188         return;
189     }
190
191     if (choice == 1)
192     {

```

```

Start here X *Medicine.c X
193     med[index].stock = med[index].stock + amount;
194     printf("Stock added.\n");
195 }
196 else if (choice == 2)
197 {
198     if (amount > med[index].stock)
199     {
200         printf("Not enough stock.\n");
201         return;
202     }
203     med[index].stock = med[index].stock - amount;
204     printf("Medicine issued.\n");
205 }
206 }
207 else
208 {
209     printf("Wrong choice.\n");
210     return;
211 }
212 }
213 showMedicine(index);
214 }
215
216 /* Sort medicines by priority */
217 void sortMedicine()
218 {
219     int i, j;
220     struct Medicine temp;
221     for (i = 0; i < n - 1; i++)
222     {
223         for (j = 0; j < n - 1 - i; j++)
224         {
225             if (priority(med[j]) < priority(med[j + 1]))
226             {
227                 temp = med[j];
228                 med[j] = med[j + 1];
229                 med[j + 1] = temp;
230             }
231         }
232     }
233     printf("\nMedicine sorted by priority.\n");
234 }
235
236 /* Save report in file */
237
238
239
240

```

```

Start here X *Medicine.c X
241 void saveFile()
242 {
243     FILE *fp;
244     int i;
245
246     fp = fopen("stock_report.txt", "w");
247
248     if (fp == NULL)
249     {
250         printf("File cannot be created.\n");
251         return;
252     }
253
254     fprintf(fp, "MEDICINE STOCK REPORT\n\n");
255
256     for (i = 0; i < n; i++)
257     {
258         fprintf(fp, "%s %s Stock:%d Coverage:%.2f Expiry:%d Priority:%d\n",
259             med[i].code,
260             med[i].name,
261             med[i].stock,
262             coverage(med[i]),
263             med[i].expiry,
264             priority(med[i]));
265     }
266
267     fclose(fp);
268
269     printf("\nReport saved successfully.\n");
270 }
271
272 /* Main menu */
273 void menu()
274 {
275     int choice;
276
277     do
278     {
279         printf("\n\n==== MEDICINE MANAGEMENT =====");
280         printf("\n1. Display Medicine");
281         printf("\n2. Search Medicine");
282         printf("\n3. Update Stock");
283         printf("\n4. Sort by Priority");
284         printf("\n5. Save Report");
285         printf("\n6. Exit");
286
287         printf("\nEnter choice: ");
288

```

```

Start here X *Medicinec X
286     printf("\n6. Exit");
287
288     printf("\nEnter choice: ");
289     scanf("%d", &choice);
290
291     switch (choice)
292     {
293     case 1:
294         display();
295         break;
296     case 2:
297         search();
298         break;
299     case 3:
300         update();
301         break;
302     case 4:
303         sortMedicine();
304         display();
305         break;
306     case 5:
307         saveFile();
308         break;
309     case 6:
310         printf("\nProgram ended.\n");
311         break;
312     default:
313         printf("\nInvalid choice.\n");
314     }
315 } while (choice != 6);
316
317 int main()
318 {
319     menu();
320     return 0;
321 }
322
323
324
325
326
327
328
329
330
331
332

```

7. Testing and Validation

Program run

Case-1

```

C:\Users\PC\AppData\Local\... X + -
===== MEDICINE MANAGEMENT =====
1. Display Medicine
2. Search Medicine
3. Update Stock
4. Sort by Priority
5. Save Report
6. Exit
Enter choice: 1

CODE    NAME          STOCK  DAILY  MIN    EXPIRY  PRIORITY
MED01   Oral Saline       120    35     80     45      75
MED02   Paracetamol      300    40     100    120     45
MED03   Insulin          60     12     50     30      80
MED04   Amoxicillin      200    25     80     20      70
MED05   Antacid          250    18     60     15      30
MED06   Cetirizine       180    15     50     90      25

===== MEDICINE MANAGEMENT =====
1. Display Medicine
2. Search Medicine
3. Update Stock
4. Sort by Priority
5. Save Report
6. Exit
Enter choice: |

```

Case-2

```

1 | #include <stdio.h>
2 | #include <string.h>
3 |
4 | #define MAX 50
5 |
6 | struct Medicine
7 | {
8 |     char co
9 |     char na
10 |    int sto
11 |    int dai
12 |    int min
13 |    int exp
14 |    int ess
15 | };
16 |
17 | struct Medi
18 | {
19 |     ["MED01
20 |     ["MED02
21 |     ["MED03
22 |     ["MED04
23 |     ["MED05
24 |     ["MED06
25 | };
26 |
27 | int n = 6;
28 |
29 | /* Calculat
30 | float cover
31 | {
32 |     if (m.d
33 |     ret
34 | }
35 | return
36 | }
37 |
38 | /* Check me
39 | void condit
40 | {
41 |     float d
42 |     float d
43 |     if (m.sto
44 |     if (m.sto
45 |
C:\Users\PC\AppData\Local\... X + -
1. Display Medicine
2. Search Medicine
3. Update Stock
4. Sort by Priority
5. Save Report
6. Exit
Enter choice: 2
Enter medicine code: MED05
Medicine Code: MED05
Medicine Name: Antacid
Stock: 250
Daily Requirement: 18
Minimum Stock: 60
Coverage: 13.89 days
Expiry Days: 15
Essentiality: 1
Condition: Expiry Attention
Priority: 30
===== MEDICINE MANAGEMENT =====
1. Display Medicine
2. Search Medicine
3. Update Stock
4. Sort by Priority
5. Save Report
6. Exit
Enter choice: |

```

Case-3

```

4 #include <string.h>
5
6 #define MAX 50
7
8 struct Medicine
9 {
10     char code[20];
11     char name[50];
12     int stock;
13     int daily;
14     int min;
15     int expiry;
16     int essential;
17 };
18
19 struct Medicine m[MAX];
20
21 int main()
22 {
23     int choice;
24     Enter choice: 3
25     Enter medicine code: MED06
26     Enter amount: 50
27     Medicine issued.
28     1. Add Stock
29     2. Issue Medicine
30     Enter choice: 2
31     Enter amount: 50
32     Medicine issued.
33     1. Display Medicine
34     2. Search Medicine
35     3. Update Stock
36     4. Sort by Priority
37     5. Save Report
38     6. Exit
39     Enter choice: |
40     if (m.stock < m.minimum && m.expiry <= 30)
41         printf("Critical");
42 }

```

Case-4

```

===== MEDICINE MANAGEMENT =====
1. Display Medicine
2. Search Medicine
3. Update Stock
4. Sort by Priority
5. Save Report
6. Exit
Enter choice: 4

Medicine sorted by priority.

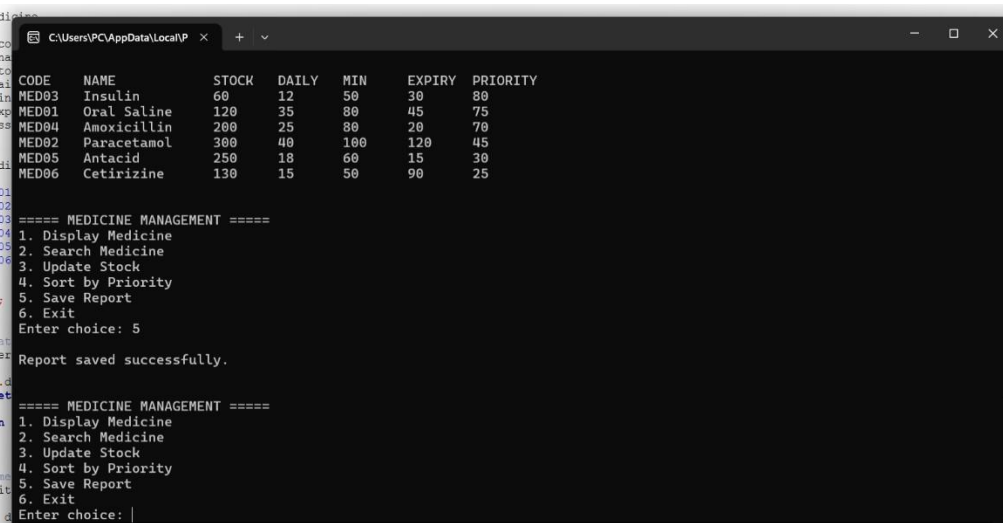
CODE    NAME           STOCK  DAILY  MIN  EXPIRY  PRIORITY
MED03   Insulin          60     12     50   30      80
MED01   Oral Saline       120    35     80   45      75
MED04   Amoxicillin       200    25     80   20      70
MED02   Paracetamol       300    40    100   120     45
MED05   Antacid           250    18     60   15      30
MED06   Cetirizine        130    15     50   90      25

===== MEDICINE MANAGEMENT =====
1. Display Medicine
2. Search Medicine
3. Update Stock
4. Sort by Priority
5. Save Report
6. Exit
Enter choice: |

```

Case-5

```
6 struct Medicine
7 {
8     char code[10];
9     char name[50];
10    int stock;
11    int daily;
12    int min;
13    int exp;
14    int ess;
15 };
16
17 struct Medicine
18 {
19     /*MED01
20     /*MED02
21     /*MED03
22     /*MED04
23     /*MED05
24     /*MED06
25 };
26
27 int n = 6;
28
29 /* Calculator
30 float cover
31 {
32     if (m.d
33     ret
34     return
35 }
36
37 /* Check me
38 void condit
39 {
40     float d
41     float d
```

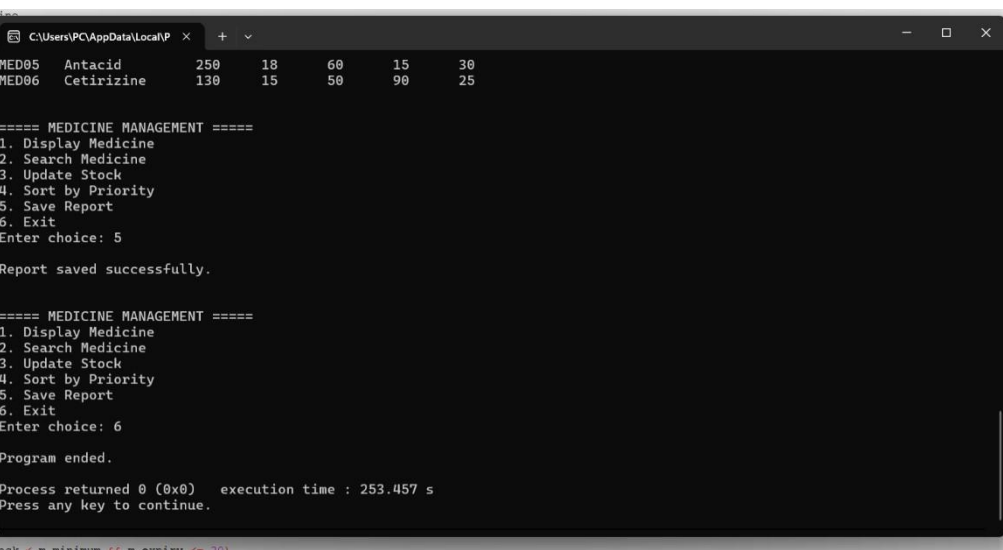


CODE	NAME	STOCK	DAILY	MIN	EXPIRY	PRIORITY
MED03	Insulin	60	12	50	30	80
MED01	Oral Saline	120	35	80	45	75
MED04	Amoxicillin	200	25	80	20	70
MED02	Paracetamol	300	40	100	120	45
MED05	Antacid	250	18	60	15	30
MED06	Cetirizine	130	15	50	90	25

```
==== MEDICINE MANAGEMENT ====
1. Display Medicine
2. Search Medicine
3. Update Stock
4. Sort by Priority
5. Save Report
6. Exit
Enter choice: 5
Report saved successfully.
==== MEDICINE MANAGEMENT ====
1. Display Medicine
2. Search Medicine
3. Update Stock
4. Sort by Priority
5. Save Report
6. Exit
Enter choice: |
```

Case-6

```
6 struct Medicine
7 {
8     char code[10];
9     char name[50];
10    int stock;
11    int daily;
12    int min;
13    int exp;
14    int ess;
15 };
16
17 struct Medicine
18 {
19     /*MED01
20     /*MED02
21     /*MED03
22     /*MED04
23     /*MED05
24     /*MED06
25 };
26
27 int n = 6;
28
29 /* Calculator
30 float cover
31 {
32     if (m.d
33     ret
34     return
35 }
36
37 /* Check me
38 void condit
39 {
40     float d
41     float d
```



CODE	NAME	STOCK	DAILY	MIN	EXPIRY	PRIORITY
MED05	Antacid	250	18	60	15	30
MED06	Cetirizine	130	15	50	90	25

```
==== MEDICINE MANAGEMENT ====
1. Display Medicine
2. Search Medicine
3. Update Stock
4. Sort by Priority
5. Save Report
6. Exit
Enter choice: 5
Report saved successfully.
==== MEDICINE MANAGEMENT ====
1. Display Medicine
2. Search Medicine
3. Update Stock
4. Sort by Priority
5. Save Report
6. Exit
Enter choice: 6
Program ended.
Process returned 0 (0x0)   execution time : 253.457 s
Press any key to continue.
```

8. Limitations

The proposed solution rests on a number of assumptions and carries corresponding limitations that should be understood when interpreting its output.

- **Stable daily requirement:** The average daily requirement for each medicine is assumed to stay reasonably constant. The program does not consider seasonal changes, disease outbreaks, or sudden increases in the number of patients that could cause demand to change.
- **Same expiry period:** The days remaining to expiry are assumed to apply to all the available stock of a medicine. The given dataset does not separate medicines into different batches with different expiry dates. If a health center has several batches with different expiry dates, the program would need to be changed to store information for each batch separately.
- **Fixed minimum stock and essentiality levels:** The minimum stock level and essentiality level are provided by the health center management and are treated as fixed values. The program does not decide these values or change them automatically.
- **Small dataset:** The program is designed for a relatively small number of medicine records. Therefore, the simple sorting method selected is suitable for the assignment. If the number of medicines becomes much larger, a different sorting method may be more suitable.
- **Chosen classification and priority method:** The classification and priority methods used in this program are one reasonable way of meeting the assignment requirements. Other methods could also be used if they are properly explained and justified. Different methods may produce slightly different priority orders while still giving generally similar results.

9. Conclusion

This assignment required the design of a structured C program for a realistic medicine inventory management problem. The program considers both shortage risk and expiry risk, while also giving more importance to medicines that are essential for patient care.

The proposed solution calculates the stock-coverage period and expiry-exposure quantity for each medicine. These values, together with the essentiality level, are used to classify medicines into five different stock conditions. A priority score is also calculated to determine which medicines need management attention first. If two medicines have the same priority, additional rules are used to decide their order.

All calculated information is kept with the correct medicine record, and only the affected medicine is recalculated when its stock is updated. The program is also designed to work with different medicine data and different numbers of records, rather than only the sample data provided.

The implementation and testing sections will be completed separately with the C source code and test evidence.

10. References

[1] Course lecture materials CSE 1383 - Structured Programming, Summer 2026.

GITHUB LINK : <https://github.com/khaled-zzz/LAB-PROJECT/upload/main>

CODE : [medicine stock.c](#)